

Université Hassan I
École Nationale des Sciences Appliquées Berrechid

RAPPORT DE TRAVAUX PRATIQUES

Gestion de Versions avec Git

Module : Maquette et Ingénierie Numérique

Réalisé par : Saad Benhaimer

Encadrant : Pr. Hrimech Hamid

Année universitaire : 2025/2026

Table des matières

1	Introduction	2
2	Initialisation et configuration d'un dépôt local	2
2.1	Création de l'architecture du projet	2
2.2	Initialisation du dépôt	2
2.3	Premier commit	2
3	Gestion des branches	2
3.1	Affichage des branches existantes	2
3.2	Création et basculement de branches	2
3.3	Travail isolé sur une branche	3
3.4	Modification parallèle sur la branche principale	3
4	Fusion de branches (Merge)	3
4.1	Fusion sans conflit	3
4.2	Vérification post-fusion	3
5	Gestion des conflits de fusion	4
5.1	Principe du conflit	4
5.2	Simulation d'un conflit	4
5.3	Tentative de fusion et détection du conflit	4
5.4	Analyse des marqueurs de conflit	4
5.5	Résolution manuelle	4
5.6	Abandon d'une fusion	5
6	Historique et comparaison	5
6.1	Visualisation de l'historique	5
6.2	Comparaison entre branches	5
6.3	Navigation dans l'historique	5
7	Nettoyage des branches	5
8	Conclusion	5

1 Introduction

Ce rapport présente les travaux pratiques consacrés à l'apprentissage de Git, l'outil de gestion de versions distribué incontournable en développement logiciel moderne. L'objectif principal est de maîtriser la création de dépôts, la manipulation des branches, la fusion de code ainsi que la résolution des conflits de fusion.

Les compétences acquises permettent de simuler un environnement collaboratif réel, où plusieurs développeurs travaillent simultanément sur un même projet sans écraser le travail des autres.

2 Initialisation et configuration d'un dépôt local

2.1 Création de l'architecture du projet

Avant d'initialiser Git, nous avons structuré notre projet en trois répertoires distincts :

- `html/` : contient la page d'accueil (`index.html`)
- `css/` : contient la feuille de styles (`style.css`)
- `js/` : contient les scripts client (`index.js`)

2.2 Initialisation du dépôt

La commande `git init` permet de transformer un dossier ordinaire en dépôt Git traçable.

```
git init
```

Résultat : création d'un sous-dossier `.git/` contenant l'historique et la configuration du projet.

2.3 Premier commit

Après avoir créé les fichiers, nous les avons indexés puis validés :

```
git add .  
git commit -m "Initial commit : structure de base html/css/js"
```

La commande `git status` confirme alors que l'arbre de travail est propre (*nothing to commit, working tree clean*).

3 Gestion des branches

3.1 Affichage des branches existantes

```
git branch
```

Par défaut, Git crée la branche `master` (ou `main` selon la configuration). L'astérisque `*` indique la branche active.

3.2 Création et basculement de branches

Pour créer une branche dédiée au développement d'une nouvelle fonctionnalité (par exemple, une refonte visuelle) :

```
# Methode 1 : creation puis basculement
git branch style
git checkout style

# Methode 2 : creation et basculement en une seule commande
git checkout -b style
```

3.3 Travail isolé sur une branche

Sur la branche `style`, nous avons modifié le fichier `css/style.css` :

```
body {
    background-color: red;
}
```

Puis nous avons validé ces changements :

```
git add css/style.css
git commit -m "[CSS] Changement de la couleur de fond en rouge"
```

En retournant sur `master` via `git checkout master`, nous constatons que le fichier `style.css` est resté dans son état initial (bleu). Cela démontre l'isolation des branches.

3.4 Modification parallèle sur la branche principale

Toujours sur `master`, nous avons modifié `html/index.html` et committé :

```
git add html/index.html
git commit -m "[HTML] Mise a jour du contenu de la page d'accueil"
```

À ce stade, `master` et `style` possèdent des historiques divergents.

4 Fusion de branches (Merge)

4.1 Fusion sans conflit

Pour intégrer le travail de la branche `style` dans `master` :

```
git checkout master
git merge style
```

Git a effectué une fusion **fast-forward** car les modifications touchaient des fichiers différents. L'historique est devenu linéaire et contient désormais les deux évolutions.

4.2 Vérification post-fusion

Le fichier `style.css` affiche désormais le fond rouge sur `master`, confirmant que la fusion a bien intégré les modifications.

5 Gestion des conflits de fusion

5.1 Principe du conflit

Un conflit survient lorsque deux branches modifient **la même zone d'un même fichier** de manière incompatible. Git ne peut pas décider automatiquement quelle version garder.

5.2 Simulation d'un conflit

Nous avons créé un fichier `readme.md` avec le contenu initial :

```
Ligne 1 : Introduction
Ligne 2 : Description
Ligne 3 : Conclusion
```

Puis nous avons créé deux branches :

- `branch-a` : modifie la ligne 1 en ajoutant des instructions spécifiques à la branche A
- `branch-b` : modifie également la ligne 1 avec des instructions spécifiques à la branche B

5.3 Tentative de fusion et détection du conflit

```
git checkout branch-a
git merge branch-b
```

Git affiche :

```
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
Automatic merge failed; fix conflicts and then commit the result.
```

5.4 Analyse des marqueurs de conflit

Le fichier `readme.md` est automatiquement modifié par Git pour montrer les divergences :

```
<<<<<< HEAD
Instructions spécifiques a la branche A
=====
Instructions spécifiques a la branche B
>>>>>> branch-b
```

- `<<<< HEAD` : version de la branche courante (`branch-a`)
- `=====` : séparateur entre les deux versions
- `>>>> branch-b` : version de la branche fusionnée

5.5 Résolution manuelle

Nous avons édité le fichier pour conserver une version fusionnée cohérente :

```
Instructions fusionnees : integration des points A et B
```

Puis nous avons finalisé la fusion :

```
git add readme.md
git commit -m "Resolution du conflit entre branch-a et branch-b"
```

5.6 Abandon d'une fusion

Si une fusion est trop complexe, il est possible de l'annuler complètement :

```
git merge --abort
```

Cette commande restaure le dépôt à son état antérieur au lancement du merge.

6 Historique et comparaison

6.1 Visualisation de l'historique

```
git log --oneline --graph --all
```

Cette commande affiche l'arbre des commits avec les branches et les points de fusion.

6.2 Comparaison entre branches

```
git diff branch-a branch-b
```

Permet d'observer les différences de contenu avant de fusionner.

6.3 Navigation dans l'historique

Il est possible de consulter un ancien commit via son hash :

```
git checkout 8523ff4
```

Le détachement de HEAD permet d'inspecter le code à un instant précis sans affecter les branches actives.

7 Nettoyage des branches

Après fusion, les branches temporaires peuvent être supprimées :

```
git branch -d branch-a  
git branch -d branch-b  
git branch -d style
```

8 Conclusion

Ce TP nous a permis d'acquérir une maîtrise opérationnelle de Git dans un contexte de développement collaboratif. Nous avons appris à :

- Initialiser et structurer un dépôt
- Créer et naviguer entre des branches isolées
- Fusionner des historiques divergents
- Identifier et résoudre manuellement des conflits de fusion
- Utiliser l'historique comme outil de traçabilité

Ces compétences constituent la base indispensable pour tout projet logiciel moderne utilisant un workflow de type Git Flow ou GitHub Flow.